

Fundamentos Da Segurança Informática

t01 - Security Concepts

1. Definição e Ferramentas Criptográficas

A segurança informática é definida como o conjunto de mecanismos destinados a **dissuadir, prevenir, detetar e corrigir violações à segurança da informação**. Para atingir este propósito, a segurança recorre a algoritmos e protocolos criptográficos, que se agrupam em quatro áreas principais:

- **Encriptação simétrica:** Utilizada para ocultar o conteúdo de blocos ou fluxos de dados de qualquer dimensão, incluindo mensagens, ficheiros, passwords e até outras chaves de encriptação.
- **Encriptação assimétrica:** Aplicada na ocultação de blocos de dados de pequena dimensão, tais como chaves de encriptação ou valores de funções de *hash* utilizados na criação de assinaturas digitais.
- **Algoritmos de integridade:** Desenvolvidos especificamente para proteger os dados (como mensagens) contra qualquer tipo de alteração indevida.
- **Protocolos de autenticação:** Consistem em esquemas baseados em criptografia com o intuito de autenticar a verdadeira identidade das entidades comunicantes.

2. Objetivos da Segurança Informática

Os pilares essenciais da segurança em redes e computadores são:

- **Confidencialidade:** Divide-se na *confidencialidade dos dados* (garantindo que informação privada não é divulgada a indivíduos não autorizados) e na *privacidade* (assegurando que as pessoas controlam que informação sua é recolhida, por quem, e a quem é divulgada).
- **Integridade:** Ramifica-se em *integridade dos dados* (garante que os dados e programas apenas são alterados de forma autorizada e especificada) e *integridade do sistema* (garante que o sistema opera livre de manipulações inadvertidas ou deliberadas).
- **Disponibilidade:** Garante que os sistemas funcionam atempadamente e que o serviço nunca é negado aos utilizadores devidamente autorizados.
- Adicionalmente, requisitos essenciais englobam também a **Autenticidade** (Authenticity) e a **Responsabilização/Auditoria** (Accountability).

3. Ameaças e Ataques à Segurança

Dois grandes *standards* guiam a abordagem a estes problemas: a norma **X.800 (ITU-T)**, que define a arquitetura estrutural da segurança respondendo à questão “O que é a segurança da informação?”, e o **RFC 4949 (IETF)**, um glossário que normaliza a terminologia, esclarecendo “Como falamos corretamente sobre segurança?”. Consoante o RFC 4949, uma **ameaça** é o mero potencial para uma violação de segurança (um perigo possível que pode explorar uma vulnerabilidade), enquanto um **ataque** é uma tentativa deliberada, derivada de uma ameaça inteligente, para contornar os serviços de segurança.

Os ataques classificam-se em duas categorias primárias:

- **Ataques Passivos:** Têm como objetivo obter informação que está a ser transmitida, mas não afetam os recursos do sistema. Exemplos incluem a leitura de emails ou chamadas VoIP não encriptadas e a análise de tráfego/captura de pacotes. Devido à enorme dificuldade de os detetar ativamente, **a ênfase no seu combate é sempre colocada na prevenção**.

- **Ataques Ativos:** Têm como propósito alterar os recursos do sistema ou afetar drasticamente o seu funcionamento normal. Uma vez que existem múltiplas vulnerabilidades (físicas, de rede e de *software*), são ataques muito difíceis de prevenir; logo, **o foco recai em detetá-los atempadamente e recuperar das perturbações**. Subdividem-se em:
 - **Repetição (Replay):** Captura passiva de uma unidade de dados e posterior retransmissão para obter efeitos não autorizados (ex: reutilizar *tokens* de sessão).
 - **Mascaramento (Masquerade):** Uma entidade usurpa a identidade de outra, muitas vezes como base para outro ataque ativo (ex: *IP spoofing*).
 - **Modificação de mensagens:** Alteração parcial, reordenação ou atraso de partes de uma mensagem legítima (ex: ataque *Man-in-the-middle*).
 - **Negação de Serviço (DoS):** Impedir ou inibir totalmente a gestão e uso normal das infraestruturas de comunicações.

4. Serviços de Segurança

Para mitigar os ataques referidos, os sistemas implementam as seguintes categorias de serviços:

- **Autenticação:** Garante que o recetor tem a certeza de quem enviou a mensagem. A norma X.800 desdobra-a em *Peer entity authentication* (garante que um “peer” é quem diz ser durante a sessão contínua) e *Data origin authentication* (garante exclusivamente a origem da mensagem isolada).
- **Controlo de Acesso:** Consiste em limitar e regular o acesso aos recursos das aplicações com base numa política de segurança rigorosa, dependendo sempre da identificação e autenticação prévias das entidades para a atribuição de permissões.
- **Confidencialidade dos Dados:** Protege os dados de ataques passivos, podendo ser global (todo um período de tempo), restrita (apenas campos específicos) ou assumir a forma de *proteção do fluxo de tráfego* (escondendo a origem, destino e frequência das mensagens).
- **Integridade dos Dados:** Protege as transmissões contra alterações. Pode ser *orientada à ligação* (protege um fluxo de mensagens contra duplicação, reordenação e repetição) ou *não orientada à ligação* (protege mensagens individuais apenas contra modificação, sem contexto alargado).
- **Não repudição (Non-repudiation):** Garante a existência de provas irrefutáveis sobre uma troca de dados, impedindo o emissor de negar que enviou e o recetor de negar que recebeu.
- **Disponibilidade:** Garante a prontidão e operacionalidade contínua dos recursos, combatendo falhas ou ataques (ex: DoS) recorrendo a medidas estruturais como a redundância e mecanismos de tolerância a falhas e recuperação.

5. Superfícies de Ataque e Modelos Conceptuais

- **Superfícies de Ataque:** Este conceito engloba todas as vulnerabilidades que podem ser alcançadas e exploradas no sistema. São exemplos as portas abertas em servidores acessíveis do exterior, os serviços disponíveis no interior da *firewall*, o processamento de código/XML/SQL de entrada ou mesmo um colaborador alvo de engenharia social.
- **Modelos de Segurança:** O documento ilustra esquematicamente como a segurança é construída, apresentando:
 - *O Modelo de Segurança de Rede*, onde as transformações criptográficas e uma Terceira Parte de Confiança tentam proteger o canal de informação de um Oponente.
 - *O Modelo de Segurança de Acesso à Rede*, onde uma função sentinela (*Gatekeeper*) é colocada no Canal de Acesso para afastar *hackers*, vírus ou *worms* dos controlos internos e processos do Sistema de Informação.

t02 - Firewalls and intrusion detection

1. Políticas e Mecanismos de Controlo de Acesso

O controlo de acesso começa com a definição de uma **Política de Acesso**, que é um dos objetivos principais da Política de Segurança de uma organização e deve ser estipulada antes da implementação de qualquer mecanismo. De seguida, aplica-se o **Controlo de Acesso** propriamente dito, que executa e obriga ao cumprimento da política, sendo habitualmente implementado lado a lado com sistemas de autenticação. Os mecanismos práticos dividem-se em:

- **Barreiras Físicas:** Paredes, portas fechadas, armários, etc.
- **Barreiras Lógicas:** Permissões, regras de acesso e Firewalls.

Para configurar o controlo de acesso, analisam-se diversos elementos: a direção do tráfego (entrada ou saída), o serviço em causa (HTTP, FTP, SMTP), o *Host* (origem ou destino), o utilizador (identificação e papel), fatores temporais (hora, dia da semana), tipo de ligação (pública ou privada) e a Qualidade de Serviço (QoS, como largura de banda e atraso).

2. Firewalls: Definição e Cenários de Utilização

Uma Firewall é um sistema ou grupo de sistemas cuja função é impor uma política de acesso. É amplamente utilizada em cenários de acesso à Internet, acessos remotos (via 3G/4G, Wi-Fi) e ligações a redes de organizações parceiras ou clientes. A Firewall controla o chamado “**Perímetro de Segurança**” através da restrição de acessos a serviços (por IP e porto), gestão da direção do tráfego, controlo de utilizadores (locais ou remotos) e inspeção de comportamento (avaliando o conteúdo para evitar ataques à camada aplicacional).

3. Tipos de Firewalls

Existem três tipos fundamentais de Firewalls na arquitetura de redes:

- **Filtros de Pacotes (Packet filters):** Operam ao nível da rede (Camada 3) e são tipicamente implementados num *router*. Utilizam Listas de Controlo de Acesso (ACLs), como as do Cisco, IPTables ou IPFW. As regras baseiam-se nos endereços IP de origem/destino, protocolo de transporte (UDP, TCP) e portos de origem/destino. Podem usar estratégias estáticas ou dinâmicas (para serviços que usam portos dinâmicos) e operar com *Network Address Translation* (NAT).
- **Gateways ao nível do circuito (Circuit-level gateways):** Operam ao nível de transporte (ex: protocolo SOCKS, RFC 1928). Funcionam interceptando as ligações TCP/UDP, verificando as permissões e, se autorizado, estabelecendo uma nova ligação com o destino, concatenando ambas as ligações. São úteis para dar acesso à Internet a *hosts* com IPs privados e suportam tráfego em tempo real para aplicações que não suportam *proxies* aplicacionais.
- **Gateways ao nível da aplicação (Application-level gateways):** Operam ao nível aplicacional, funcionando como *proxies* (ex: SQUID para HTTP, ftpgw para FTP). Servem os pedidos dos utilizadores de acordo com as suas permissões e reencaminham a informação entre o cliente e o servidor remoto. Têm a vantagem adicional de poder fazer *cache* dos pedidos, poupando largura de banda.

4. Filtro de Pacotes em Linux (IPTables/NetFilter)

O documento detalha o uso do **IPTables**, que permite filtrar pacotes em fases específicas de processamento do Kernel Linux (denominadas *chains* ou cadeias), agrupadas em tabelas:

- **Tabela *filter*:** Usada para filtragem padrão com as *chains* INPUT (tráfego de entrada), OUTPUT (tráfego de saída) e FORWARD (tráfego reencaminhado).

- **Tabela *nat*:** Modifica pacotes para tradução de endereços, usando as *chains* PREROUTING (antes do encaminhamento), OUTPUT e POSTROUTING (antes de sair do sistema).
- **Tabela *mangle*:** Usada para alterar cabeçalhos IP.

5. Configurações e Topologias de Firewalls

A localização geométrica e a topologia das firewalls variam em três configurações padrão:

- **Screened host (Bastion host):** Baseia-se no uso de um “Bastion host”, um servidor altamente protegido que usa uma versão segura do Sistema Operativo, corre apenas serviços vitais e controla as comunicações do perímetro. Pode ter uma ou duas interfaces de rede (single-homed ou dual-homed).
- **Screened subnet:** Implementa três níveis de defesa: a rede externa, a Zona Desmilitarizada (DMZ) e a rede segura/interna. O *Bastion host* e os servidores de serviços públicos (como o servidor Web ou de Email) são colocados na DMZ. As comunicações são restritas: o exterior só acede à DMZ, e a rede interna pode aceder à DMZ. O tráfego entre estas três zonas é estritamente controlado por filtros de pacotes, habitualmente instalados em dois *routers* (um externo e um interno).

6. Sistemas de Detecção de Intrusões (IDS)

Os IDSs são ferramentas concebidas para identificar, avaliar e reportar atividades não autorizadas numa rede. Monitorizam o tráfego em tempo real para detetar padrões de ataque através de regras, heurísticas ou Inteligência Artificial. O seu principal objetivo é complementar a Firewall (que foca em prevenir) focando-se em detetar ataques novos, vírus, *trojans* e anomalias de conteúdo interno. A arquitetura padrão engloba o *Detection engine* (captura e analisa) e a *Console* (gera relatórios e alarmes). Existem três variantes principais de distribuição:

- **Host-based IDS (HIDS):** Instalados em servidores críticos. Analisam os recursos locais da máquina, como as chamadas do Sistema Operativo, eventos aplicacionais e integridade dos ficheiros de *logs* (Exemplo: OSSEC). Verificam a eficácia de um ataque, mas têm impacto nos recursos da máquina.
- **Network-based IDS (NIDS):** Dispositivos dedicados espalhados pela rede que inspecionam o tráfego em tempo real (fazendo *sniffing* de pacotes TCP/IP). Têm um espectro de visão mais alargado sobre toda a rede e são independentes dos Sistemas Operativos (Exemplo: SNORT).
- **Hybrid IDS:** Sistemas que combinam HIDS e NIDS numa única plataforma (Exemplo: Prelude SIEM).

7. Abordagens de Detecção e Problemas dos IDSs

As técnicas para distinguir tráfego legítimo de malicioso recaem em dois modelos teóricos:

- **Signature-based IDS:** Procuram por sequências de *bytes* que correspondam exatamente a uma “assinatura” ou lista de ataques já conhecidos. São simples de perceber, mas **são cegos a novos ataques** para os quais ainda não existem assinaturas criadas.
- **Anomaly-based IDS:** Estabelecem uma linha de base (baseline) do que é o comportamento “normal” da rede. Se houver desvios estatísticos dessa norma, geram um alarme. Têm a vantagem de detetar novos ataques, mas sofrem de altas taxas de falsos alarmes.

A operação de um IDS enfrenta sempre dois dilemas fundamentais:

- **Falsos Negativos:** O IDS falha na deteção de um ataque (muito enganador pois a administração nem se apercebe que a rede foi comprometida).
- **Falsos Positivos:** O IDS rotula tráfego perfeitamente normal como malicioso (gera o chamado “Falso Alarme”, obrigando o administrador a refinar as regras constantemente para evitar o bloqueio de serviços válidos).

8. Implementação de captura de tráfego

Para um NIDS funcionar, tem de interceptar os dados da rede. Isso é feito através do *Mirroring ports* (espelhar portas de um *Switch*, como o SPAN nos Cisco), utilizando *Networking TAPs* (hardware físico inserido na cablagem para copiar fibra ou cobre) ou forçando a repetição de pacotes utilizando *Hubs*.

t03 - Data integrity

1. Comparação entre Hash, MAC e Assinaturas Digitais

O documento inicia-se estabelecendo a diferença fundamental entre as três principais primitivas criptográficas utilizadas para garantir a integridade da informação:

- **Hash:** Não utiliza chaves (*unkeyed*). O seu foco é proteger os dados contra alterações acidentais. Garante **Integridade**, mas não fornece Autenticação nem Não-repudição.
- **MAC (Message Authentication Code):** Trata-se de um *hash* com chave (*keyed hash*), utilizando **chaves simétricas** (partilhadas). Protege as mensagens contra a falsificação por qualquer entidade que não conheça o segredo. Garante **Integridade** e **Autenticação**, mas não assegura Não-repudição.
- **Assinatura Digital:** Recorre a **chaves assimétricas** (pública e privada). Garante em pleno todos os requisitos de segurança neste contexto: **Integridade**, **Autenticação** e **Não-repudição**.

2. Funções de Hash

Uma função de *hash* criptográfica (H) aceita um bloco de dados de tamanho variável (M) como entrada e produz um valor de *hash* de tamanho fixo ($h = H(M)$). O seu objetivo primário é a **integridade dos dados**, permitindo determinar se uma mensagem foi ou não alterada. Para ser considerada criptograficamente segura, tem de ser computacionalmente inviável (exceto por força-bruta):

- Dada uma saída específica, encontrar um objeto de entrada correspondente (**Propriedade one-way**).
- Encontrar dois objetos de entrada completamente diferentes que resultem exatamente no mesmo *hash* (**Propriedade collision-free**).

Operação Geral:

A mensagem original é sujeita a um *padding* (preenchimento) para se tornar num múltiplo inteiro de um bloco de tamanho fixo. Este preenchimento inclui invariavelmente o comprimento da mensagem original em *bits*, uma técnica que aumenta substancialmente a dificuldade de execução de ataques.

Exemplo Prático: MD5

- Desenhado por Ronald Rivest em 1991 (RFC 1321), produz um *hash* fixo de 128 *bits* (em mensagens processadas em blocos de 512 *bits*).
- Usa registos de 32 *bits* (A, B, C, D) e cada bloco passa por 64 rondas de operações bit-a-bit (AND, OR, XOR), rotações e adições (procurando o “Efeito de Avalanche”).
- **Estado atual:** É extremamente rápido, mas **estruturalmente fraco**. Sofre de vulnerabilidades a ataques de colisão. Devido às suas falhas de segurança, **não é recomendado** para autenticação ou criptografia, servindo apenas para verificações simples de integridade não-segura (*checksums*).

Evolução para o SHA (Secure Hash Algorithm)

Criado pelo NIST (FIPS 180), o padrão SHA é a evolução moderna e segura:

- **SHA-1 (1995):** Produz *hashes* de 160 *bits*. Também considerado **inseguro e quebrado** face a colisões desde 2005.
- **SHA-2 (2001):** Utiliza a estrutura Merkle-Damgård e oferece vários tamanhos (SHA-224, SHA-256, SHA-384, SHA-512). É atualmente considerado **forte e seguro**, sendo o padrão de mercado para certificados TLS/SSL e Blockchains.
- **SHA-3 (2015):** Usa uma construção diferente chamada *Sponge-based* (Keccak). É extremamente forte e resistente a colisões, concebido à prova do futuro, embora ligeiramente mais lento que o SHA-2.

Outros Casos de Uso dos Hashes:

- **Ficheiros de senhas unidirecionais:** Como o `/etc/shadow` no Linux (que recorre tipicamente ao SHA-512), permitindo validar *passwords* sem conhecer o seu conteúdo original em texto limpo.
- **Deteção de intrusões e vírus:** Calculando e protegendo o $H(F)$ de ficheiros cruciais do sistema para detetar alterações não autorizadas.
- **Funções e Geradores Pseudoaleatórios (PRF e PRNG):** Criação de algoritmos cuja saída é impossível de distinguir da aleatoriedade genuína para quem não conheça a chave partilhada/semente.

3. Autenticação de Mensagens e MAC

Um recetor necessita de ter a certeza absoluta da origem da mensagem. O documento demonstra várias construções utilizando algoritmos de *hash*:

- Cifrando a mensagem + *hash* com uma chave simétrica.
- Cifrando apenas o *hash*.
- Concatenando a mensagem com um “segredo partilhado” (S) e executando um *hash* sobre ambos (com ou sem encriptação simétrica global).

No entanto, a forma mais padronizada de o fazer é através do **MAC (Message Authentication Code)**, que é um valor de *hash* dependente de uma **chave secreta simétrica** partilhada entre remetente e destinatário. Como apenas ambas as partes possuem a chave secreta, se o cálculo final coincidir com o MAC recebido, prova-se não apenas que os dados estão íntegros, mas também a sua proveniência autorizada. Exemplos estruturais englobam o **HMAC** (usando SHA), AES-CMAC e GMAC.

O Padrão HMAC (RFC 2104 / FIPS 198-1):

É a construção mais popular. Os seus principais objetivos de desenho incluem:

- **Ausência de modificações:** Utiliza as funções de *hash* existentes (como o SHA-256) diretamente, sem necessidade de alterar o seu código.
- **Substituição fácil:** O algoritmo de *hash* subjacente pode ser trocado imediatamente no caso de se descobrir uma falha (ex: trocar HMAC-MD5 por HMAC-SHA256).
- **Desempenho:** Mantém a performance nativa do algoritmo de *hash*.
- **Casos de Uso:** É amplamente usado para a autenticação de APIs (como no AWS), ligações TLS/SSL, túneis IPSec e em *JSON Web Tokens* (JWT).

4. Assinaturas Digitais

Enquanto o MAC usa chaves simétricas, as Assinaturas Digitais baseiam-se na **Criptografia Assimétrica** para atingir o nível máximo de segurança e resolver o problema da não-repudição. Neste processo, **o valor de *hash* da mensagem é cifrado estritamente com a chave privada** do autor. Qualquer pessoa pode posteriormente usar a chave pública do autor para decifrar esse *hash* e verificar a sua validade contra o documento recebido.

Requisitos e Propriedades:

- Tem de verificar o autor, a data e a hora da assinatura e autenticar o conteúdo do documento naquele preciso momento temporal.
- A assinatura é um padrão de *bits* que depende intrinsecamente do remetente e da mensagem. Tem de ser fácil de produzir, fácil de validar por **terceiros** (para resolução de disputas) e computacionalmente impraticável de forjar ou de transferir para outra mensagem.

Algoritmos Abordados:

- **NIST DSA (Digital Signature Algorithm):** Padronizado no FIPS 186. Tem como base o *Problema do Logaritmo Discreto* (DLP). Apresenta assinaturas muito rápidas a gerar, mas mais lentas a validar.
- **RSA:** Tem como base o *Problema da Fatorização de Inteiros* (IFP). Tem assinaturas mais lentas a gerar, mas muito rápidas a validar.
- **ECC (Elliptic Curve Cryptography):** Baseia-se no *Problema do Logaritmo Discreto em Curvas Elípticas* (ECDLP). É globalmente mais eficiente, extremamente rápido tanto na assinatura como na verificação, e fornece segurança equivalente aos demais algoritmos usando um tamanho de chave absurdamente mais curto (ex: ECC de 256 *bits* equivale a um RSA de 3072 *bits*).

t04 - Symmetric Encryption

1. Conceitos Fundamentais e Modelos de Sistemas Criptográficos

O documento inicia estabelecendo a diferença entre a encriptação assimétrica (que usa chaves diferentes para cifrar e decifrar, resolvendo o problema de distribuição de chaves) e a **encriptação simétrica**, que é muito mais rápida, mas utiliza a mesma chave para cifrar e decifrar, exigindo mecanismos seguros *out-of-band* para a distribuição dessa chave. A terminologia base inclui o **Texto Limpo** (*Plaintext*), o **Texto Cifrado** (*Ciphertext*), a **Encriptação** (processo de converter o texto limpo em cifrado) e a **Desencriptação** (o inverso). O ecossistema estuda-se através da **Criptografia** (criação de esquemas) e da **Criptoanálise** (técnicas para decifrar sem conhecer os detalhes de encriptação).

Os sistemas criptográficos caracterizam-se por três dimensões:

- **O tipo de operações:** **Substituição** (elementos do texto são mapeados noutros) ou **Transposição** (a ordem dos elementos é rearranjada).
- **O número de chaves:** Simétrico (uma chave) ou Assimétrico (duas chaves).
- **O processamento do texto:** **Cifras de Bloco** (processam um bloco de elementos de cada vez) ou **Cifras de Fluxo** (processam continuamente um elemento de cada vez).

Existem duas abordagens principais para atacar a encriptação simétrica: a **Criptoanálise** (que explora as características do algoritmo e conhecimentos prévios sobre o texto limpo) e o **Ataque de Força-Bruta** (que testa todas as chaves possíveis até obter um texto inteligível).

2. Técnicas Clássicas de Encriptação

- **Técnicas de Substituição:** Substituem letras por outras. O exemplo mais simples é a **Cifra de César**, que desloca o alfabeto em 3 posições. Este método monoalfabético é muito vulnerável a ataques de força-bruta (pois só tem 25 chaves possíveis) e à **análise de frequência** (comparando a frequência das letras cifradas com a distribuição estatística do idioma original, ex: a letra 'E' em Inglês).
- **Cifras Polialfabéticas:** Utilizam múltiplas regras de substituição monoalfabéticas ao longo da mensagem. A mais conhecida é a **Cifra de Vigenère**, que usa uma palavra-passe repetida para determinar o

deslocamento de cada letra, tornando a análise de frequência mais difícil. Uma variação é o sistema *Autokey*, que concatena a palavra-passe com a própria mensagem.

- **Técnicas de Transposição:** Alteram a ordem original. Um exemplo é a cifra *Rail Fence*, onde o texto é escrito em diagonais e lido em linhas.
- **Máquinas de Rotores:** Máquinas complexas baseadas em cilindros que rodam, criando cifras polialfabéticas com um período massivo (ex: a máquina Enigma).
- **Esteganografia:** Em vez de tornar a mensagem ininteligível, o objetivo é **esconder a existência da mensagem** em si (ex: tinta invisível ou manipulação dos *bits* menos significativos nos píxeis de uma imagem digital).
- **One-Time Pad:** É o único sistema matematicamente inquebrável e com “sigilo perfeito” (*perfect secrecy*). Exige uma chave aleatória do mesmo tamanho da mensagem, usada uma única vez e depois destruída. A sua limitação colossal reside na extrema dificuldade de distribuição e geração de imensas chaves verdadeiramente aleatórias.

3. Cifras de Bloco

A maioria das aplicações modernas usa **Cifras de Bloco** (geralmente blocos de 64 ou 128 *bits*) baseadas na estrutura idealizada por Horst Feistel, que alterna substituições e permutações de forma iterativa. Este design procura garantir dois princípios introduzidos por Claude Shannon:

- **Difusão (Diffusion):** Dissipar as estatísticas estruturais do texto limpo ao longo de todo o texto cifrado (um *bit* do texto limpo deve afetar muitos *bits* do texto cifrado).
- **Confusão (Confusion):** Tornar a relação entre as estatísticas do texto cifrado e a chave o mais complexa possível.

4. Modos de Operação das Cifras de Bloco

Para aplicar algoritmos de bloco a dados de qualquer dimensão, o NIST definiu 5 modos principais de operação:

- **ECB (Electronic Codebook):** Os blocos são cifrados isoladamente com a mesma chave. É inseguro para dados estruturados, pois blocos de texto limpo idênticos geram blocos de cifra idênticos (falta de difusão), permitindo inferir padrões (como se nota ao cifrar uma imagem bitmap).
- **CBC (Cipher Block Chaining):** O bloco de texto limpo sofre XOR com o bloco de texto cifrado imediatamente anterior antes de ser encriptado. O primeiro bloco usa um **Vetor de Inicialização (IV)** aleatório. Esconde padrões perfeitamente, mas a encriptação tem de ser feita de forma sequencial (não paralelizável). É fulcral garantir que o Vetor de Inicialização (IV) nunca é reutilizado com a mesma chave, prevenindo padrões que comprometeriam a segurança.
- **CFB (Cipher Feedback):** Converte a cifra de bloco numa cifra de fluxo.
- **OFB (Output Feedback):** Converte a cifra de bloco numa cifra de fluxo, gera a cifra independentemente do texto e é resistente a erros de transmissão na linha.
- **CTR (Counter Mode):** Cifra o valor sucessivo de um contador. É altamente eficiente, totalmente paralelizável e não sofre de propagação de erro de blocos, sendo excelente para transferências de alta velocidade e sistemas IoT.

5. Rede de Feistel

Na Rede de Feistel, o bloco de entrada é dividido em duas metades iguais (Esquerda e Direita). Ao longo de várias “rondas”, aplica-se uma função complexa F à metade direita em conjunto com uma subchave. O resultado sofre a operação XOR com a metade esquerda, e por fim, ambas as metades trocam de posição para a ronda seguinte. O facto de o mesmo algoritmo servir para cifrar e decifrar (bastando inverter a ordem das subchaves) facilita imenso a sua implementação em *hardware* e *software*. A segurança desta rede

depende do tamanho do bloco, tamanho da chave, número de rondas e da complexidade da função de geração de subchaves.

6. Algoritmos Simétricos de Bloco: DES, 3DES e AES

- **DES (Data Encryption Standard):** Standard de 1977. Cifra blocos de 64 *bits* com uma **chave de apenas 56 bits**, usando 16 rondas de Feistel. A sua função de ronda expande a metade direita para 48 *bits*, faz XOR com a chave, e passa o resultado pelas **S-Boxes** (tabelas de substituição não lineares cruciais para a segurança) e **P-Boxes** (permutações). Tem o desejado **Efeito de Avalanche** (alterar um *bit* no texto limpo altera dezenas de *bits* no cifrado). O DES foi descontinuado devido ao tamanho muito curto da sua chave, tornando-o suscetível a ataques de força-bruta (quebrável em minutos).
- **3DES (Triple DES):** Surgiu como solução transitória, aplicando o DES três vezes seguidas com duas ou três chaves diferentes (Encripta - Decrypta - Encripta). O uso de 3 chaves fornece um tamanho de chave efetivo de 168 *bits*.
- **AES (Advanced Encryption Standard):** O standard atual que substituiu o DES. Tem um bloco de 128 *bits* e tamanhos de chave de 128, 192 ou 256 *bits* (o que lhe dá 10, 12 ou 14 rondas de processamento, respetivamente). **Não utiliza a rede de Feistel**. É imensamente resistente a ataques de força-bruta e de criptoanálise diferencial/linear, e também altamente eficiente em qualquer plataforma.

7. Geração de Bits Aleatórios e Cifras de Fluxo (*Stream Ciphers*)

- **PRNG vs TRNG:** Geradores Verdadeiramente Aleatórios (TRNG) usam fontes físicas de entropia. Geradores Pseudoaleatórios (PRNG e PRF) usam algoritmos determinísticos e uma semente (*seed*) inicial para gerar fluxos de *bits* que parecem aleatórios.
- **Cifras de Fluxo:** Usam um PRNG para gerar uma “corrente” de chaves (*keystream*) que sofre um XOR *bit a bit* ou *byte a byte* com a mensagem. Requerem grandes períodos de repetição, distribuição uniforme de zeros e uns e chaves de pelo menos 128 *bits*. São incrivelmente velozes e requerem muito pouco código, sendo ideais para canais de comunicações contínuos (ex: Wi-Fi, páginas Web).
- **RC4:** Um exemplo clássico de *stream cipher* criado por Ron Rivest. É focado no *byte*, usa um vetor de permutação de tamanho 256 que vai rodando as suas componentes para gerar o fluxo. É tão simples e rápido em *software* que foi o pilar de standards de rede originais como o WEP, WPA e SSL/TLS.

t05 - Assymetric ciphers

1. A Revolução da Criptografia de Chave Pública e Princípios Básicos

A criptografia de chave pública representa um desvio radical na história da criptografia. Ao contrário dos sistemas simétricos, que se baseiam em operações complexas de substituição e permutação, a criptografia assimétrica utiliza funções matemáticas e opera com **duas chaves separadas (um par)**.

Um esquema de encriptação de chave pública possui 6 ingredientes fundamentais:

1. **Texto Limpo (Plaintext):** A mensagem original legível.
2. **Algoritmo de Encriptação:** Realiza transformações matemáticas sobre o texto limpo.
3. **Chave Pública (Public key):** Utilizada para a encriptação (ou desencriptação).
4. **Chave Privada (Private key):** Utilizada para a encriptação (ou desencriptação).
5. **Texto Cifrado (Ciphertext):** A mensagem embaralhada produzida.
6. **Algoritmo de Desencriptação:** Recupera o texto limpo a partir do texto cifrado e da chave correspondente.

Para que o sistema funcione e seja seguro, tem de obedecer aos seguintes requisitos:

- O remetente e o destinatário devem cada um possuir uma chave do par correspondente (mas não a mesma).
- Tem de ser computacionalmente impraticável determinar a chave de descriptação conhecendo apenas o algoritmo e a chave de encriptação.
- **Uma das chaves tem de ser mantida estritamente secreta.**
- Tem de ser impraticável decifrar a mensagem se a chave complementar for mantida em segredo.

2. Aplicações de Sistemas de Chave Pública

Os criptosistemas de chave pública classificam-se em três grandes categorias de aplicação:

- **Encriptação/Desencriptação (Confidencialidade):** O remetente encripta a mensagem utilizando a **chave pública do destinatário**. Garante que apenas o destinatário (que possui a chave privada) consiga ler a mensagem.
- **Assinatura Digital (Autenticação e Integridade):** O remetente “assina” a mensagem utilizando a sua **própria chave privada**. Por questões de eficiência de armazenamento e computação, geralmente não se assina a mensagem inteira, mas sim um resumo (*Hash/MAC*) da mesma. Isto garante a origem, pois ninguém consegue alterar a mensagem sem possuir a chave privada do autor.
- **Troca de Chaves (Key Exchange):** Duas partes cooperam para trocar de forma segura uma **chave de sessão** (uma chave secreta que será depois usada num sistema de encriptação simétrica durante um curto período).

3. Criptoanálise e Funções Trap-door

Tal como os sistemas simétricos, a criptografia de chave pública é vulnerável a ataques de força-bruta. A contramedida óbvia é usar chaves de tamanho muito grande. No entanto, devido à complexidade matemática, usar chaves gigantes torna a encriptação e desencriptação lentas demais para uso geral. É por este motivo que, na prática, **a criptografia assimétrica está confinada a assinaturas digitais e gestão de chaves** (frequentemente conjugada com a criptografia simétrica, como no sistema PGP).

A base matemática destes sistemas é a **Função Unidirecional Trap-door**:

- $Y = f_k(X)$ é fácil de calcular se a “trapdoor” k e X forem conhecidos.
- $X = f_k^{-1}(Y)$ é fácil de calcular se k e Y forem conhecidos.
- $X = f_k^{-1}(Y)$ é **computacionalmente inviável** se Y for conhecido, mas k não.

4. O Algoritmo RSA

Ao contrário dos sistemas simétricos que precisam de geradores de números aleatórios, o RSA envolve puramente matemática, usando expressões com exponenciais. O texto limpo é encriptado em blocos, onde cada bloco tem um valor menor que um número n .

A segurança do RSA baseia-se na **extrema dificuldade em fatorizar grandes números inteiros**.

A “trapdoor” é o conhecimento da chave privada d , que necessita do conceito da **Função Totiente de Euler** $\phi(n)$. Esta função conta os inteiros positivos até n que são coprimos de n (cujo máximo divisor comum é 1). Se conhecermos a fatorização de n em dois números primos (p e q), calcular a função é extremamente simples: $\phi(n) = (p - 1) \times (q - 1)$.

Fase de Setup do RSA (Geração do par de chaves):

1. Escolher dois números primos enormes, p e q (na prática moderna, pelo menos 2^{512} , ou seja, de 1024 bits de comprimento cada).

2. Calcular $n = p \times q$.
3. Calcular o totiente de Euler: $\phi(n) = (p - 1) \times (q - 1)$.
4. Escolher a chave de encriptação pública e a partir do conjunto $\{1, 2, \dots, \phi(n) - 1\}$, garantindo que o máximo divisor comum entre e e $\phi(n)$ é 1 (são relativamente primos).
5. Calcular a chave privada d , que será o inverso multiplicativo de e módulo $\phi(n)$. Ou seja, $d \times e \equiv 1 \pmod{\phi(n)}$.

Fase de Operação:

- **Chave Pública:** (e, n)
- **Chave Privada:** (d, n)
- **Encriptação:** O texto cifrado é calculado como $C = M^e \pmod{n}$.
- **Desencriptação:** A mensagem original recupera-se como $M = C^d \pmod{n}$.

O documento exemplifica o processo com números pequenos (ex: $p = 11, q = 3$, logo $n = 33, \phi(33) = 20$. Escolhendo $e = 3$, temos $d = 7$, pelo que a chave pública é $\{3, 33\}$ e a privada $\{7, 33\}$. Encriptar $M = 7$ dá $C = 13$, que desencriptado devolve o 7). Outro exemplo é resolvido com $p = 17, q = 11$.

5. Garantias de Segurança do RSA

O atacante tem acesso ao texto cifrado, a e e a n . Para encontrar d , o atacante teria obrigatoriamente de calcular $\phi(n)$. Mas, para calcular $\phi(n)$, precisa de descobrir quais são os fatores primos p e q que compõem n . Este processo de fatorização de inteiros para um número de 1024 bits (cerca de 300 dígitos) é, hoje em dia, impossível na prática. Como termo de comparação, o atual recorde mundial de fatorização usando algoritmos de propósito geral ocorreu em 2020 e apenas conseguiu fatorizar o RSA-250 (um número semiprimo de “apenas” 829 bits / 250 dígitos).

6. Mitos e Falsas Ideias

- *A encriptação assimétrica é mais segura que a simétrica?* Não. Como em qualquer sistema, a segurança depende pura e exclusivamente do tamanho da chave e do trabalho computacional exigido para quebrar a cifra.
- *Tornou a encriptação simétrica obsoleta?* Não. A encriptação assimétrica é excessivamente lenta, servindo sobretudo para apoiar e proteger a gestão de chaves do mais eficiente mundo simétrico.
- *A distribuição de chaves públicas é trivial?* É muito mais fácil de gerir, mas ainda assim exige algum protocolo de validação rigoroso para garantir a autenticidade das chaves trocadas.

t06 - Key Management and User Authentication

1. Técnicas de Distribuição de Chaves Simétricas

A criptografia simétrica exige que as duas partes partilhem a mesma chave secreta e que esta seja protegida de terceiros. A mudança frequente de chaves é desejável para limitar a quantidade de dados comprometidos caso a chave seja descoberta. A distribuição de chaves entre A e B pode ser feita de várias formas:

- A seleciona uma chave e entrega-a fisicamente a B.
- Uma terceira parte seleciona a chave e entrega fisicamente a A e a B.
- Se A e B já tiverem uma chave antiga, uma parte pode transmitir a nova chave encriptada usando essa chave antiga.
- Se A e B tiverem, cada um, uma ligação encriptada a uma terceira parte C (um Centro de Distribuição de Chaves - KDC), C pode entregar a chave a A e B através dessas ligações seguras.

Se não existir um KDC e a encriptação for feita ponto-a-ponto (nível de rede/IP), uma rede com N nós exigirá $[N(N - 1)]/2$ chaves. Por exemplo, uma rede com 1000 nós precisaria de distribuir cerca de meio milhão de chaves. Para resolver este problema de escalabilidade, utiliza-se uma **Hierarquia de Chaves**: usam-se *Session Keys* (chaves de sessão temporárias para os dados) que são distribuídas de forma segura com o auxílio de *Master Keys* (chaves mestre de longa duração partilhadas com o KDC).

2. O Modelo de Autenticação Kerberos

O Kerberos é um serviço de autenticação de terceiros que utiliza chaves simétricas e opera em três fases para autenticar clientes de forma segura:

1. **Pedido do TGT (Ticket-Granting Ticket):** O cliente autentica-se no *Authentication Server* (AS) e recebe um TGT encriptado.
2. **Pedido ao TGS (Ticket-Granting Server):** O cliente apresenta o TGT e pede acesso a um serviço específico. O TGS valida e fornece um *Ticket* de serviço.
3. **Acesso ao Serviço:** O cliente apresenta o *Ticket* ao Servidor aplicacional, garantindo **autenticação mútua** através de carimbos de tempo (*timestamps*) e provas de conhecimento da chave de sessão.

O documento também demonstra como o Kerberos opera entre diferentes domínios (*cross-realm*), permitindo que um cliente de um “Realm 1” obtenha um ticket para um servidor num “Realm N”. No que toca ao **tempo de vida das chaves de sessão**, existe um balanço: quanto mais frequentemente as chaves forem trocadas, mais seguras são, exigindo protocolos adequados caso a comunicação seja orientada (nova chave por sessão) ou não orientada à ligação.

3. Distribuição de Chaves Simétricas com Criptografia Assimétrica

O uso desprotegido de chaves públicas para trocar chaves secretas pode resultar num ataque passivo muito simples ou num ataque *Man-in-the-Middle* (MitM), onde um atacante como o “Darth” interceta a comunicação, fazendo-se passar por Bob para a Alice e vice-versa. Para evitar isto, é apresentado um protocolo de **Distribuição de Chaves Secretas com Confidencialidade e Autenticação**. Este modelo assume que A e B já conhecem as chaves públicas um do outro e usa **Nonces** (números aleatórios usados apenas uma vez) em 4 passos rigorosos para identificar unicamente a transação e proteger o intercâmbio contra ataques de repetição (*replay attacks*).

4. Formas de Distribuição de Chaves Públicas

Existem várias abordagens para distribuir chaves públicas:

- **Publicação da Chave Pública (Public Announcement):** Qualquer um envia a sua chave ou publica num espaço comum. É perigoso, pois é muito vulnerável a forja (qualquer pessoa pode fazer-se passar por outra).
- **Diretório Disponível Publicamente:** O registo é feito pessoalmente sob alguma forma de autenticação e mantido por uma entidade de confiança. É vulnerável se a chave privada do diretório for comprometida.
- **Autoridade de Chaves Públicas:** Garante maior controlo. Usa-se um protocolo complexo de 7 passos com *nonces* (como N_1 e N_2) e carimbos de tempo para comunicar com a Autoridade e obter de forma robusta e fresca a chave pública de outra entidade.
- **Certificados de Chave Pública:** A abordagem moderna. Usa um certificado que associa a identidade do utilizador à sua chave pública, assinado pela chave privada de uma Autoridade de Certificação (CA) confiável.

5. Norma X.509 e Cadeias de Certificação

A norma estrutural padrão para certificados de chave pública é a **X.509**. Usa criptografia assimétrica e assinaturas digitais (geralmente RSA). Um certificado contém a chave pública do dono e está assinado pela CA emissora.

- **Cadeias de Certificados:** Se a Alice tem um certificado emitido pela CA_1 e o Bob tem um emitido pela CA_2 , a Alice pode comprovar o certificado do Bob se existir uma cadeia de confiança. A CA_1 assina um certificado para a CA_2 ($X_1 \ll X_2 \gg$), permitindo à Alice confiar na CA_2 e validar o Bob ($X_2 \ll B \gg$).
- Isto forma uma **Hierarquia X.509** em árvore, assente numa *Trust Anchor* (o Certificado Raiz). Os browsers e sistemas operativos vêm equipados com listas de CAs bem conhecidas e confiáveis para fazer esta validação automática de *sites* HTTPS.

6. Revogação de Certificados (CRL, OCSP e Stapling)

Os certificados têm um período de validade, mas às vezes precisam de ser anulados prematuramente (ex: chave privada comprometida, trabalhador deixou a empresa, CA foi comprometida).

- **CRL (Certificate Revocation List):** Lista mantida pela CA com todos os certificados revogados mas ainda não expirados. É estática e exige que os clientes a descarreguem.
- **OCSP (Online Certificate Status Protocol):** Protocolo de internet que permite obter o estado de revogação de um certificado de forma interativa e *online* via HTTP/HTTPS, sem descarregar toda a CRL. O URL é frequentemente incluído no próprio certificado X.509.
- **OCSP Stapling:** Para evitar sobrecarregar os servidores da CA com os pedidos constantes via OCSP dos clientes, o próprio servidor *web* obtém uma resposta OCSP (com carimbo de tempo e assinada pela CA) e “agrafá-a” (*staple*) durante o aperto de mãos (Handshake TLS) inicial, entregando logo a prova de validade ao browser.

7. Modelo Arquitetural PKIX

A PKI (Infraestrutura de Chaves Públicas) engloba o hardware, software, pessoas, políticas e procedimentos necessários para criar, gerir, armazenar, distribuir e revogar certificados digitais. O grupo PKIX definiu um modelo rigoroso que engloba as seguintes funções de gestão para os protocolos PKI: Registo, Inicialização, Certificação, Recuperação de par de chaves, Atualização de chaves, Pedidos de revogação e Certificação cruzada (*cross certification*).

t07 - Web and Transport-Level Security

1. Considerações e Ameaças à Segurança Web

A World Wide Web é fundamentalmente uma aplicação cliente/servidor que opera sobre TCP/IP. A necessidade de ferramentas de segurança desenhadas à medida para a Web deriva de várias características: os servidores Web são fáceis de configurar e o conteúdo fácil de desenvolver; no entanto, o *software* subjacente é extraordinariamente complexo e pode ocultar falhas de segurança. Um servidor Web pode ser explorado como uma plataforma de lançamento de ataques para toda a rede corporativa, e os clientes são, muitas vezes, utilizadores casuais e sem treino em cibersegurança, desconhecendo os riscos e as contramedidas.

As ameaças na Web dividem-se em quatro categorias:

- **Integridade:** Modificação de dados do utilizador, de memória ou de tráfego em trânsito (ex: *Trojans* no *browser*). Causa perda de informação e comprometimento da máquina. Pode ser mitigada com *checksums* criptográficos.

- **Confidencialidade:** Escuta de rede (*eavesdropping*) e roubo de informação de clientes, servidores ou sobre a configuração da rede. Resulta em perda de privacidade. Combate-se com encriptação e *proxies* Web.
- **Negação de Serviço (DoS):** Inundação com pedidos falsos (*bogus requests*), exaustão de memória/disco ou ataques de isolamento de DNS. Causa disrupção e é bastante difícil de prevenir.
- **Autenticação:** Falsificação de dados e usurpação da identidade de utilizadores legítimos. Gera uma falsa crença na validade da informação. Pode ser evitada através de técnicas criptográficas.

2. Localização da Segurança na Pilha TCP/IP

A segurança pode ser implementada em várias camadas:

- **Nível de Rede:** Utilizando IP/IPSec (transparente, mas a um nível inferior).
- **Nível de Transporte:** Utilizando o SSL ou o TLS operando logo acima do TCP, para proteger protocolos como o HTTP, FTP e SMTP.
- **Nível Aplicacional:** Extensões de segurança embebidas nas próprias aplicações, como o S/MIME, Kerberos, etc.

3. Transport Layer Security (TLS) e a sua Arquitetura

O TLS é um dos serviços de segurança mais amplamente utilizados, padronizado pelo RFC 5246, e que evoluiu a partir do protocolo comercial Secure Sockets Layer (SSL). É um serviço de propósito geral que pode ser incluído em pacotes de *software* ou correr de forma transparente para as aplicações sob o TCP.

A arquitetura do TLS assenta em dois conceitos cruciais:

- **Ligação TLS (TLS connection):** Um transporte efémero *peer-to-peer*. Todas as ligações estão associadas a uma sessão.
- **Sessão TLS (TLS session):** Uma associação lógica entre cliente e servidor, criada através do *Handshake Protocol*. A sessão define um conjunto de parâmetros de segurança criptográficos que podem ser partilhados e reutilizados entre múltiplas ligações contínuas, **evitando assim a negociação dispendiosa de novos parâmetros** sempre que se abre uma nova ligação.
- O “Estado da Ligação” inclui parâmetros como *Nonces* aleatórios (Server e Client random), chaves secretas de MAC, chaves de encriptação, Vetores de Inicialização (IVs) e números de sequência.

A pilha do protocolo está dividida em duas camadas:

1. **TLS Record Protocol:** A camada inferior que garante os serviços básicos de segurança (Confidencialidade via encriptação simétrica e Integridade via MAC) para os protocolos superiores. A sua operação faseada consiste em: pegar nos dados da aplicação -> fragmentar -> comprimir -> adicionar o MAC -> encriptar -> anexar o cabeçalho (Header) do Registo SSL.
2. **Protocolos de Gestão de topo:** Incluem o *Handshake Protocol*, *Change Cipher Spec Protocol* e o *Alert Protocol*.

4. O Protocolo de Handshake e os Cálculos Criptográficos

O *Handshake Protocol* estabelece as capacidades de segurança através de várias mensagens (como *client_hello*, *server_hello*, *certificate*, etc.). O cerne deste processo é a criação do **Master Secret** partilhado, um valor único de 48 *bytes* para a sessão.

Para chegar ao *Master Secret*, os sistemas passam por duas etapas:

1. **Geração e Troca do Pre-master Secret:**

- **Via RSA:** O servidor envia o seu certificado com a chave pública RSA. O cliente valida o certificado, gera o *pre-master secret* (48 bytes aleatórios), encripta-o com a chave pública do servidor e envia-o de volta. Como só o servidor detém a chave privada RSA para o decifrar, isto **autentica implicitamente o servidor**.
 - **Via Diffie-Hellman Efêmero (DHE):** O servidor escolhe os parâmetros DH e envia-os assinados pela sua chave. O cliente valida a assinatura e envia o seu parâmetro DH. Ambos calculam o mesmo segredo partilhado. A grande vantagem do DHE é a **Confidencialidade Perfeita (Forward Secrecy)**: como as chaves DH são descartadas após o *handshake*, se a chave de longo termo do servidor for comprometida no futuro, as sessões passadas continuam seguras.
2. **Derivação do Master Secret:** O *pre-master secret* nunca é usado isoladamente. Ele é misturado, através de uma Função Pseudoaleatória (PRF baseada em HMAC), com dois *Nonces* (o *ClientHello.random* e o *ServerHello.random*) para gerar o *master secret* de 48 bytes.
- *Porquê misturar com números aleatórios?* Garante que cada parte contribui com entropia fresca, prevenindo ataques de repetição (*replay attacks*). Mesmo que o *pre-master* fosse o mesmo, os números aleatórios garantem um *master* sempre diferente.
 - Finalmente, o *master secret* volta a passar pela PRF para derivar todas as chaves operacionais (chaves MAC, chaves de encriptação e IVs) e nunca é usado diretamente para encriptar tráfego.

5. Protocolo Heartbeat e HTTPS

- **Heartbeat (RFC 6250):** É um sinal periódico (extensão do TLS) para monitorizar a disponibilidade de uma entidade/serviço e manter a ligação sincronizada e viva.
- **HTTPS:** Resulta pura e simplesmente de correr os pedidos e respostas do protocolo HTTP protegidos dentro do SSL/TLS.

6. Protocolo Secure Shell (SSH)

O SSH (padronizado no RFC 4251) é um protocolo da camada de transporte desenhado para logins remotos e transferências seguras. Pode usar dois modelos de confiança para autenticar a identidade do servidor (chave do *host*): uma base de dados local gerida pelo cliente ou validação através de uma Autoridade de Certificação (CA).

A formação de um pacote SSH envolve pegar no *payload*, comprimi-lo, adicionar tamanho, *padding*, encriptar este conjunto e anexar um MAC. Para autenticação do utilizador do lado do cliente, o SSH suporta o envio da chave pública assinada pela chave privada do cliente, ou o uso padrão de *Password*.

7. Port Forwarding e Canais SSH

Após o túnel SSH estar estabelecido, o *Connection Protocol* multiplexa várias sessões (canais lógicos) independentes. Existem 4 tipos de canais: *Session* (execução remota de programas), *X11* (interfaces gráficas em rede), *Forwarded-tcpip* e *Direct-tcpip*.

O **Port Forwarding (SSH Tunneling)** é uma das funcionalidades mais vitais do SSH. Permite converter qualquer ligação TCP originalmente insegura e encapsulá-la dentro do túnel seguro do SSH. Pode ser executado de duas formas principais:

- **Local Port Forwarding (ssh -L):** Os pacotes direcionados a um porto lógico da máquina local do utilizador são transportados com segurança pelo túnel até chegarem à máquina remota, sendo depois reencaminhados para um servidor alvo (ex: contornando firewalls para aceder a serviços não expostos).
- **Remote Port Forwarding (ssh -R):** A máquina remota abre um porto e reencaminha o tráfego que lá chega, de volta pelo túnel seguro, para a máquina local do utilizador.

t08 - IP Security

A segurança ao nível do IP (Camada de Rede na pilha TCP/IP) fornece uma plataforma de rede segura transversal a todas as aplicações. A arquitetura IPsec (*IP Security*) engloba mecanismos de **autenticação, confidencialidade e gestão de chaves** para garantir a segurança da infraestrutura de rede contra monitorização não autorizada e para proteger o tráfego end-to-end entre os utilizadores.

1. Especificações e Documentos do IPsec

O ecossistema IPsec é complexo e encontra-se dividido num amplo conjunto de documentos normativos (RFCs), sendo os principais:

- **Arquitetura:** Cobre conceitos gerais, requisitos e mecanismos (RFC 4301).
- **Encapsulating Security Payload (ESP):** Fornece encriptação ou uma combinação de encriptação com autenticação (RFC 4303).
- **Authentication Header (AH):** Fornece autenticação de mensagens e integridade (RFC 4302).
- **Internet Key Exchange (IKE):** Descreve esquemas de gestão e troca de chaves criptográficas, principalmente através da norma IKEv2 (RFC 7296).
- **Algoritmos Criptográficos:** Conjunto de documentos que detalham a encriptação, funções pseudoaleatórias (PRFs) e autenticação de mensagens.

2. Protocolos de Segurança: AH e ESP

O IPsec baseia-se em dois protocolos fundamentais, que operam de formas distintas:

- **AH (IP Authentication Header):** Focado puramente na integridade e autenticação dos pacotes IP. **Não suporta confidencialidade** (não encripta os dados). Uma vez que o AH verifica o pacote IP inteiro (incluindo o cabeçalho original), qualquer alteração ao pacote faz com que a verificação de integridade falhe, o que significa que o **AH não pode coexistir com sistemas NAT** (*Network Address Translation*).
- **ESP (IP Encapsulated Security Payload):** Fornece integridade, autenticação e **confidencialidade** (encriptação dos dados). Como o ESP foca a sua autenticação apenas no encapsulamento e não no cabeçalho IP externo, **pode coexistir com NAT**.

3. Modos de Operação: Transporte vs Túnel

Tanto o AH como o ESP podem operar em dois modos de rede:

- **Modo de Transporte:** Fornece proteção sobretudo a protocolos de camadas superiores (como um segmento TCP, UDP ou ICMP). É tipicamente usado para **comunicações end-to-end (diretas) entre dois hosts**. Neste modo, o cabeçalho IPsec é inserido entre o cabeçalho IP original e o cabeçalho TCP/UDP superior.
- **Modo de Túnel:** Fornece proteção ao **pacote IP inteiro**. O pacote IP original é completamente encapsulado (e encriptado no caso do ESP) dentro de um novo pacote, com um **novo cabeçalho IP exterior**. É habitualmente usado para cenários de **Redes Privadas Virtuais (VPNs)** em ligações gateway-a-gateway ou rede-a-rede. Este modo esconde os endereços IP internos, protocolos e portas, mas consome maior largura de banda devido à adição do novo cabeçalho IP.

4. Associações de Segurança (SA) e Arquitetura Lógica

O núcleo da política de segurança do IPsec reside na criação e gestão das **Security Associations (SA)**. Uma SA é uma **ligação lógica unidirecional** entre um emissor e um recetor; logo, para uma comunicação bidirecional

completa, são exigidas no mínimo duas SAs. Numa transmissão, uma SA é univocamente identificada por três parâmetros:

1. **Security Parameters Index (SPI):** Um número inteiro de 32 *bits* que identifica a SA de forma local.
2. **IP Destination Address:** O endereço do nó final da associação (pode ser o host final ou um *router/firewall*).
3. **Security Protocol Identifier:** Indica se a associação aplica o protocolo AH ou o ESP.

Para aplicar e gerir esta arquitetura, os sistemas dependem estritamente da interação entre duas bases de dados:

- **Security Association Database (SAD):** Define os parâmetros de cada associação, armazenando o SPI, contadores de números de sequência, janela anti-replay, algoritmos de autenticação/criptação (com as respetivas chaves e IVs), modo de protocolo (túnel ou transporte) e o tempo de vida (*lifetime*) da associação, para decidir quando deve ser renovada.
- **Security Policy Database (SPD):** É o mecanismo que define que políticas aplicar ao tráfego IP. Usa **seletores** (valores específicos de filtragem como IP de origem/destino, protocolo de camada superior, portos locais/remotos) para intercetar o tráfego de saída e mapeá-lo para uma SA específica no SAD. O pacote filtrado pode sofrer três ações: ser descartado (**DISCARD**), passar livremente sem segurança (**BYPASS**) ou ser protegido pelo IPSec (**PROTECT**).

5. Processamento de Pacotes e Mecanismos Anti-Replay

- No **tráfego de saída (Outbound)**, o sistema consulta a SPD com base nos dados do pacote; se a política ditar “**PROTECT**”, procura a SA correspondente na SAD, invoca a troca de chaves se a SA ainda não existir (via IKE), processa a encriptação/autenticação e envia o pacote para a rede.
- No **tráfego de entrada (Inbound)**, o sistema identifica que é um pacote IPSec, procura na SAD qual é a associação baseada no SPI, processa a desencriptação/autenticação, cruza a validade daquele tráfego com a SPD e entrega à camada aplicacional superior.
- Como o protocolo IP é instável, o IPSec implementa um **Mecanismo Anti-Replay** baseado numa janela deslizante. Se um pacote autenticado chegar “à esquerda” (atrasado demais ou repetido) dessa janela, o pacote é imediatamente descartado para evitar ataques de repetição e é registado um evento de auditoria.

6. Combinação de SAs e IKE

Como uma SA individual apenas implementa ou AH ou ESP (não ambos simultaneamente), as SAs podem ser agregadas em **Security Association Bundles**. Estas podem ser combinadas através de:

- *Transport adjacency:* Aplicação de múltiplos protocolos de segurança ao mesmo pacote, sem recorrer a túneis.
- *Iterated tunneling:* Aplicação de múltiplos níveis de segurança inter-dependentes, forçando níveis de encapsulamento em túnel sucessivos.
- Finalmente, a gestão e distribuição segura destas chaves é gerida pelo **Internet Key Exchange (IKE)**, dado que a arquitetura IPSec necessita muitas vezes do cálculo contínuo de até 4 chaves operacionais diferentes para assegurar a comunicação segura entre duas partes.

t09 - Electronic Mail Security

1. Arquitetura de Email na Internet e Protocolos

A arquitetura de correio eletrónico na Internet está definida no RFC 5598 e é composta por vários módulos funcionais. O autor da mensagem utiliza um **MUA (Message User Agent)**, ou cliente de email, que interage com a infraestrutura. A mensagem passa por agentes de submissão (MSA), transferência (MTA) e entrega (MDA) até chegar ao armazenamento (MS) do destinatário. Existem dois tipos fundamentais de protocolos de email:

- **Protocolos de Transferência:** Utilizados para mover as mensagens através da Internet entre servidores (MTAs). O principal é o **SMTP (Simple Mail Transfer Protocol)**, um protocolo cliente-servidor baseado em texto que encapsula a mensagem num “envelope”. Originalmente definido no RFC 821 em 1982, é hoje mais comumente utilizado na sua versão estendida, o **ESMTP (Extended SMTP)**, cuja última revisão data de 2008 (RFC 5321).
- **Protocolos de Acesso:** Utilizados pelo MUA para aceder às caixas de correio e mensagens armazenadas nos servidores. Destacam-se o **POP3 (Post Office Protocol)**, que permite descarregar as mensagens via ligação TCP e apagá-las do servidor, e o **IMAP (Internet Message Access Protocol)**, que fornece capacidades avançadas de visualização e gestão das mensagens diretamente no servidor.

2. Ameaças e Extensões de Segurança Base

O documento demonstra, através da análise de cabeçalhos, a extrema facilidade com que é possível forjar o endereço de origem (“From”) de uma mensagem de email. Para combater estas ameaças, são aplicadas várias camadas de segurança:

- **STARTTLS:** Uma extensão do SMTP que fornece confidencialidade, integridade, autenticação e não-repudição a toda a ligação SMTP, correndo o protocolo sobre TLS.
- **S/MIME e PGP:** Ao contrário do STARTTLS (que protege o canal), estes mecanismos garantem a segurança end-to-end do próprio **corpo da mensagem**.

3. Pretty Good Privacy (PGP)

O PGP é um protocolo alternativo de segurança de email criado por Phil Zimmerman em 1991, que se tornou extremamente popular para uso pessoal por ter sido disponibilizado de forma gratuita. Hoje em dia, a sua versão normalizada é o **OpenPGP (RFC 4880 e 3156)**. A sua relevância cultural e prática é até ilustrada pelo seu uso no caso de Edward Snowden (documentário *Citizenfour*) para comunicações seguras. Existem duas diferenças cruciais entre o PGP e o S/MIME:

- **Certificação de Chaves:** Enquanto o S/MIME usa o padrão rígido X.509 gerido por autoridades, o OpenPGP baseia-se num modelo descentralizado de “**Teia de Confiança**” (**Web of Trust**).
- **Distribuição de Chaves:** O S/MIME recorre a Autoridades de Certificação (CA), ao passo que o OpenPGP usa **Servidores de Chaves Públicas** (como o *SKS OpenPGP Key server*).

A norma NIST 800-177 recomenda o uso do S/MIME em detrimento do PGP, precisamente devido à maior confiança oferecida pelo sistema formal de CAs na validação de chaves públicas.

4. DNS, DNSSEC e DANE

O **DNS (Domain Name System)** é o serviço de diretoria essencial que mapeia nomes de *hosts* para endereços IP. No contexto do email, os MTAs consultam o DNS para descobrir os registos “MX” (Mail Exchange) do domínio de destino. Sendo um protocolo originalmente inseguro, foram criadas extensões:

- **DNSSEC (DNS Security Extensions):** Protege os clientes DNS de aceitarem registos forjados ou alterados, fornecendo autenticação de origem e verificação de integridade através de **assinaturas digitais**. O administrador de zona assina digitalmente cada conjunto de registos, e a confiança é estabelecida hierarquicamente a partir de uma zona de raiz confiável (a *trust anchor*).
- **DANE (DNS-Based Authentication of Named Entities):** Proposto no RFC 6698, é um protocolo que permite vincular certificados X.509 a nomes de domínio usando a segurança do DNSSEC. O grande objetivo do DANE é **substituir a dependência cega no sistema de Autoridades de Certificação (CAs)** pela infraestrutura de confiança atestada pelo DNSSEC.

5. Sender Policy Framework (SPF)

O SPF é um protocolo padronizado que resolve um dos maiores problemas que causa o *spam*: o facto de qualquer *host* poder usar o domínio de outra entidade nos campos “MAIL FROM” ou “HELLO”. O SPF permite que o domínio remetente declare publicamente (através de registos TXT no seu DNS) **quais são os endereços IP ou hosts autorizados a enviar emails em seu nome**. O servidor que recebe o email pode verificar esta política antes mesmo de processar o conteúdo da mensagem. O registo SPF usa mecanismos e modificadores:

- **Mecanismos:** ip4, ip6 (especificam endereços), mx (autoriza os *hosts* listados nos registos MX), include (pesquisa recursiva noutro domínio) e all (corresponde a todos os restantes IPs).
- **Modificadores:** + (Aprova - oposição predefinida), - (Rejeita/Não autorizado), ~ (Transição/Soft fail - aceita mas marca para inspeção) e ? (Neutro - trata como se o SPF não dissesse nada).

6. DomainKeys Identified Mail (DKIM)

O DKIM atua como um complemento ao SPF. É uma especificação que permite assinar as mensagens de correio eletrónico criptograficamente. Isto permite ao domínio emissor clamar responsabilidade irrefutável sobre a mensagem, assegurando adicionalmente que o conteúdo validado não sofreu alterações em trânsito.

t10 - Web application security

1. O que é uma Aplicação Web e a sua Superfície de Ataque

Uma aplicação web é um software que corre num servidor remoto e é acedido através de um *browser* web (que atua como cliente). Estas aplicações são alvos preferenciais de ataques porque, por desenho, são publicamente acessíveis, lidam frequentemente com dados sensíveis (credenciais, pagamentos, informações pessoais) e são construídas por equipas grandes com recurso a múltiplos componentes de terceiros. Atualmente, a exploração de vulnerabilidades e o abuso de credenciais são os principais vetores iniciais de acesso em violações confirmadas.

A **Superfície de Ataque** de uma aplicação engloba todos os seus pontos de entrada (inputs), nomeadamente:

- **Formulários HTML:** Login, registo, pesquisa, etc..
- **Query Strings / URL:** Parâmetros visíveis e diretamente manipuláveis.
- **Cabeçalhos HTTP:** Muitas vezes não validados (ex: User-Agent, Referer, Cookie, X-Forwarded-For).
- **APIs REST / GraphQL:** Corpos JSON/XML e parâmetros de rota.
- **Upload de Ficheiros:** Vetores para *malware* e travessia de diretórios (*path traversal*).
- **WebSockets:** Canais persistentes que contornam proteções baseadas na ausência de estado (*stateless*).

O próprio formato do **pedido HTTP** atua como vetor de ataque:

- O cabeçalho Host pode sofrer injeção (ex: envenenamento de cache ou redefinição de password para domínio do atacante).

- O User-Agent foi o vetor do *Log4Shell* (acionando execução remota de código - RCE).
- O cabeçalho Cookie é alvo de roubo de sessão ou CSRF, muitas vezes por falta de *flags* de segurança (HttpOnly, Secure, SameSite).
- A Authorization sofre falhas como aceitação de *tokens* JWT expirados ou sem assinatura (alg:none).
- O X-Forwarded-For permite contornar limites de taxa (rate limit) baseados no IP.
- Os corpos em JSON/XML são alvos de Injeção de SQL (SQLi), NoSQLi e Entidades Externas XML (XXE).

2. Regras de Ouro da Segurança Web

Para proteger a aplicação, devem aplicar-se regras vitais:

- **NUNCA confiar no cliente.**
- **TODA a validação de input tem de ser feita do lado do servidor** (rejeitar no servidor, e não apenas no cliente, pois o JavaScript pode ser contornado).
- **Whitelist > Blacklist:** Definir estritamente o que é permitido e bloquear tudo o resto por omissão.
- **Codificar no output (*Encode on output*):** Previne ataques XSS, adaptando a codificação (HTML, JS, URL) ao contexto para que o *browser* não o interprete como código.
- **Parametrizar *queries*:** Nunca concatenar o input diretamente em comandos SQL/LDAP/OS.
- **Privilégio mínimo:** Processos e contas só devem ter as permissões estritamente necessárias.

3. O Projeto OWASP e o Top 10 de Riscos

A OWASP (*Open Web Application Security Project*) é a referência global para a segurança aplicacional. O documento destaca vários projetos como o **OWASP ZAP** (scanner de vulnerabilidades usado em CI/CD), o **WSTG** (Guia de Testes) e o **OWASP Top 10**, uma lista atualizada dos riscos mais críticos baseada em dados reais.

As 10 categorias de risco principais (com exemplos práticos de ataques) são:

- **A01 - Broken Access Control (Quebra no Controlo de Acessos):** É a vulnerabilidade mais comum. Ocorre quando o servidor verifica se o utilizador está autenticado, mas não se tem autorização para o recurso específico.
 - *Exemplo prático: IDOR (Insecure Direct Object Reference).* Um atacante altera um ID num URL (ex: `/api/users/42/orders` para 43) e acede aos dados de outros clientes sem gerar alertas nos *logs*, extraindo bases de dados inteiras. *Mitigação:* Exigir sempre verificação de autorização baseada na propriedade do recurso do lado do servidor.
- **A02 - Cryptographic Failures (Falhas Criptográficas):** Dados sensíveis expostos devido a criptografia fraca, obsoleta ou chaves expostas (ex: em ficheiros do GitHub).
 - *Exemplo prático:* Armazenar senhas com algoritmos descontinuados (MD5/SHA-1) e sem *salt*. Um atacante com uma placa gráfica moderna consegue quebrar 164 mil milhões de *hashes* MD5 por segundo, descobrindo as senhas em milissegundos. *Mitigação:* Usar Argon2id ou bcrypt, e obrigar a ligações HTTPS.
- **A03 - Injection (Injeção):** Input não confiável é interpretado como código.
 - *Exemplo de SQLi:* Concatenar input num login. Injetar `admin'--` ou `admin' OR '1'='1` permite contornar a autenticação ou fazer um *dump* de dados com UNION SELECT.
 - *Exemplo de XSS (Cross-Site Scripting):* Ocorre em três variantes — **Refletido** (a carga maliciosa no URL reflete-se na resposta HTTP), **Armazenado/Stored** (o input malicioso é guardado numa base de dados e executado nos *browsers* de todas as vítimas que visitem a página) e **Baseado em DOM** (a vulnerabilidade existe inteiramente no *browser* do lado do cliente, quando o JS pega num dado não fiável e o escreve no DOM de forma insegura). O XSS permite o roubo de *cookies* de sessão.

Mitigação para a Injeção: Parametrizar *queries* SQL, utilizar codificação atenta ao contexto no output e implementar Políticas de Segurança de Conteúdo (CSP).

- **A04 - Insecure Design (Desenho Inseguro):** Falhas de arquitetura que não se resolvem apenas por código.
 - *Exemplo prático:* Ausência de limites de taxa de submissão (*rate limiting*). Permite a enumeração de utilizadores através de respostas de erro despadronizadas e execução de *brute-force* contra códigos PIN/OTP, descobrindo 10 mil combinações de um PIN em apenas 10 segundos. *Mitigação:* Limitar tentativas (ex: bloqueio após 5 falhas), usar CAPTCHAs.
- **A05 - Security Misconfiguration (Má Configuração de Segurança):** Inclui as configurações por omissão.
 - *Exemplo prático:* Servidor web expõe a listagem de diretórios ou ficheiros sensíveis como .env contendo credenciais ou *backups*. Outro erro muito frequente é manter painéis administrativos (como o phpMyAdmin) expostos para o exterior utilizando credenciais padrão (root:root). *Mitigação:* Desativar *directory listing* e alterar todos os valores por omissão.
- **A06 - Vulnerable & Outdated Components:** Risco decorrente da utilização de software desatualizado.
- **A07 - Identification & Authentication Failure:** Gestão de sessão deficiente. Exemplo abordado via uso de JWT com o parâmetro alg:none, causando contorno de autenticação.
- **A08 - Software & Data Integrity Failures:** Ataques à cadeia de fornecimento ou manipulações de *software*.
 - *Exemplo prático:* Ataque ao pacote npm event-stream, onde a conta do mantenedor foi comprometida para injetar o pacote flatmap-stream, cujo código ofuscado roubava chaves privadas de carteiras Bitcoin.
- **A09 - Security Logging & Monitoring Failures (Falhas de Monitorização):** A falta de registos permite que os ataques durem, em média, 207 dias até serem detetados.
 - *Exemplo prático:* Se os *logs* não possuírem alertas face a picos de falhas (um ataque *Low and Slow* fica invisível) ou aceitarem input não sanitizado, os atacantes podem injetar novas linhas de texto forjadas (ex: usando \n no *username*) para eliminar pistas e criar um falso álibi. *Mitigação:* Sanitarizar *logs*, utilizar SIEM e métricas rígidas de deteção.
- **A10 - Server-Side Request Forgery (SSRF):** Enganar o servidor para fazer pedidos à sua própria rede interna.
 - *Exemplo prático:* O acesso abusivo ao Serviço de Metadados na nuvem (ex: AWS no IP 169.254.169.254). O atacante explora uma funcionalidade lícita de *preview* de URLs e direciona-a para esse IP interno de forma a extrair credenciais temporárias do IAM, ganhando controlo total do ambiente *cloud* (um ataque notório que causou a falha na Capital One). *Mitigação:* Validar URLs, utilizar *allowlists* e bloquear redes internas à camada de rede.

4. WSTG e Identificadores de Segurança

O documento conclui com a padronização e nomenclatura das falhas de segurança:

- O **WSTG (Web Security Testing Guide)** da OWASP é um guia sistemático com 91 casos de testes divididos por 12 categorias (como Information Gathering, Authentication Testing, API Testing, etc.). Cada caso possui um formato de identificador WSTG-CATEGORY-NN.
- Para gerir este ecossistema, os sistemas e ferramentas (como o OWASP ZAP) usam e mapeiam simultaneamente diferentes identificadores para a mesma falha (uma progressão de classificação do tipo **CWE -> CAPEC -> CVE -> OWASP / CVSS**):
 - **WSTG:** Diz **como testar** o problema.
 - **OWASP Top 10:** Fornece comunicação de risco (A01, A02, etc.).
 - **CVE:** Identifica uma falha única no mundo real que requer uma *patch*.

- **CWE:** Enumera a causa raiz no código/desenho informático (a Fraqueza).
- **CAPEC:** Explica o padrão de exploração que um atacante usa contra a fraqueza (o Padrão de Ataque).
- **CVSS:** Uma pontuação numérica baseada na gravidade, usada para priorizar a reparação.